

System Tuning

Supporting the streaming of 21+ uncompressed 16-bit mono video streams through network, disk, system memory, and GPU memory with low latency requires tuning of system parameters and device drivers. These streams are much higher throughput than usually handled by an operating system, so default values and approaches cannot keep up.

Sufficient hardware

Achieving full-rate capture, replay, and rendering requires high-performance hardware systems. The following systems were found to be sufficient:

- **Rendering:**
 - **Development laptop:** Using a local copy of the *Core Module using the Camera Simulator* running on localhost and talking to the Render Module, an Alienware m18 R1 laptop with a 13th Gen Intel(R) Core(TM) i9-13900HX, 32GB of RAM, and an NVIDIA GeForce RTX 4080 Laptop GPU (12 GB) is usually able to smoothly display 21 cameras (under Linux or Windows). This requires the DisplayWindow to have focus and sometimes requires one or two restarts of the Render Module. This system can *very slowly* (because of insufficient disk speed) run the *Storage Module* talking via localhost to a Render Module, using the pause function (space bar) often to let the storage catch up.
 - **Deployment laptop:** An Alienware 18 Area-51 laptop with an NVIDIA GeForce RTX 5090 24 GB and two Crucial 4TB T500 PCIe 4.0 x4 M.2 Internal SSDs can run *Storage Module* at full rate locally writing to a Render Module at full rate from a 21-camera recording. This can display to the local display and to an Elbit XSight HMD. This is much smaller and lighter than the desktide Render Server used as the primary development platform. It is also capable of doing depth estimation on a 25-camera system, but the speed of this is still being studied. Using these drives, it can store from a 21-camera live system at full rate until the drives are approximately halfway full; we're still investigating the use of commercial-grade drives to enable full-speed storage to full drive capacity. Using an ATTO Technology ThunderLink NS 5102 Thunderbolt 5 to 100Gb Ethernet Adapter, the system can *stream and display live data* from a 21-camera system or played back from a remote Storage Module under Linux (the Windows networking stack does not allow full-rate operation). This can also drive both a local display and an XSight.
 - **Development desk-sides:** The main development computers were large desk-side systems custom built for the project. Both use a Mellanox PCI 100GigE network connector with two SFP modules. The *Storage Server* includes a PCI four 2-TB NVMe drive enclosure with commercial-grade cards, supporting up to half an hour of full-rate capture on a 21-camera ball. The *Render Server* includes an NVIDIA GeForce RTX 4090 with 24GB. It also includes the Thunderbolt ports required to connect to a Varjo XR-4 HMD.

Supported operating systems and windowing systems

Version 3.54 of Render Module (the first version using GLAD rather than GLEW) has been tested on **Windows 11** and on **Ubuntu 22.04 - 26.04 using X11 or Wayland**.

Build in Release mode

When configuring CMake to build all repositories (ASDP_Core_API along with others), there is a very larger (factor of 10) difference in speeds between Debug and RelWithDebInfo modes and a smaller factor of improvement in speed between RelWithDebInfo and Release mode.

All repositories should be compiled with **CMAKE_BUILD_TYPE set to Release** for codes that are going to be used for performance testing or actual use. Using RelWithDebInfo is useful when chasing down crashes or profiling, and Debug is useful when chasing deeper bugs where all local variables are needed.

BIOS update and settings

The Render Server has an available BIOS update: https://www.asus.com/us/motherboards-components/motherboards/workstation/pro-ws-wrx80e-sage-se-wifi-ii/helpdesk_bios?model2Name=Pro-WS-WRX80E-SAGE-SE-WIFI-II

The BIOS update from 9/4/2024 (1602) was applied using the USB in-BIOS approach (the Windows BIOS updater built into the MyASUS app from the Microsoft store did not have the installation capabilities that were described in the online instructions, it only had a way to download the driver).

Overclocking: The new BIOS has an option under *Advanced/AMD Overclocking* to set the *Precision Boost Overdrive* to **Enabled** and to set *Gaming Mode* to **Enabled**. On one system, the BIOS claimed that no settings had been changed when *Save changes and quit* was selected, but coming back into the BIOS editor showed that they had persisted.

Windows 11 settings:

- Turn off *Enhance pointer precision* on the *Settings* mouse control panel.
- For the Alienware laptop Render Server when it has internal NVMe drives that will run the storage module, as of 1/21/2016 you must enable the native NVMe driver. The **following instructions did not work** on 1/21/2026 on the Alienware laptop: Change five registry settings (adding three values to a new key and then adding two new keys:
 - o HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Policies\Microsoft\FeatureManagement\Overrides
 - Add DWORDS set to 1 for values 735209102, 1853569164, 156965516
 - o HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\SafeBoot\Minimal\{75416E63-5912-4DFA-AE8F-3EFACCAFFB14}
 - o HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\SafeBoot\Network\{75416E63-5912-4DFA-AE8F-3EFACCAFFB14}

When running on a laptop under Linux, be sure to **disable the integrated/hybrid/Optimus Intel-based graphics** in the BIOS to avoid problems with GPU memory allocation for CUDA and for rendering. For the Alienware laptop Render Server to have wired networking: ***sudo apt install r8125-dkms*** (needed driver). For the GeForce 5090 card in the Alienware laptop, only the nVidia drivers with the name **-open** at the back of them will work; the proprietary drivers cannot recognize this card.

Application priority tuning

To ensure smooth operation with the Varjo, the Render Module must be run with high priority. This can be done from the Windows command line using the following:

```
start "" /high "C:\Program Files (x86)\ASDP_Render_Module\bin\ASDP_Render_Module.exe" [command line arguments]
```

Display tuning

When driving the Samsung 8K display, both Windows 11 and Linux can drive at **7680x4320 at 59.94 Hz**. Windows 11 can also drive the display at that resolution at 60Hz.

When driving one particular HD display, Linux can drive 1920x1080 (which we'll only display 1280x1024 of the pixels on) at both 60 and 59.94 Hz. Windows 11 is also able to display at both 60 and 59.94 on this monitor.

To make the IR camera inputs synchronized to the displays, minimizing latency over all frames, we'll plan to **drive both displays in both operating systems at 59.94Hz and sync the cameras at that rate when possible.**

When using the **8K display on Linux**, text scrolling in a window on another display attached to the same computer or moving of windows on the second display can cause tearing in the lower-left and lower-right corners of a full-screen window rendering on the 8K display. A second full-screen window on the primary display does not seem to cause this issue.

System power tuning

On Windows, the following settings should be made to ensure that the CPU and GPU maintain highest performance even when the window displaying the graphics does not have focus:

- In Windows settings under *Control Panel\Hardware and Sound\Power Options*, set the power plan to **High performance**.
- In the nVidia control panel under *3D Settings/Manage 3D settings* in the *Global Settings* tab, set *Background Application Max Frame Rate* to **Off**.
- In the nVidia control panel under *3D Settings/Manage 3D settings* in the *Global Settings* tab, set *Power management mode* from normal to **Prefer maximum performance**.

On Linux, similar settings must be made and the system suspend must be disabled so that it does not shut things down in the middle of a demonstration or long run. This requires the following:

- Run *Settings/NVIDIA X Server Settings* and use the *PowerMizer* control panel to set *Preferred mode* to **Prefer Maximum Performance**. This can also be done on the command line using:
 - o `nvidia-settings -a '[gpu:0]/GPUPowerMizerMode=1'`
 - o To make this happen automatically, do the following:
 - Add the following lines to a new file named `~/.xprofile`

```
/usr/bin/nvidia-settings -a '[gpu:0]/GPUPowerMizerMode=1'
```
 - Make it executable:


```
chmod +x ~/.xprofile
```
- Edit `/etc/systemd/sleep.conf` and uncomment the following lines in the [Sleep] section, setting their values to **no**: *AllowSuspend*, *AllowHibernation*, *AllowSuspendThenHibernate*, *AllowHybridSleep*.
- When logged in, use the *Settings/Light Locker settings* and pull the *Blank Screen After* slider to the left to set it to **Never**. Then log out and back in again. This should only need to be done once for each user.

Disk tuning

Direct writes: When writing sequentially to LLVM drives in hardware RAID configuration, the internal operating system buffers slow down the system. On Linux, it is possible to specify that file data be directly read from the client buffers by passing the `O_DIRECT` flag to the `open()` system call when opening the file. **Note:** When using direct write, the number of bytes written in any write must be a multiple of the block size (512 for our disks). This makes its use problematic because the last block in a file must include zero padding, which will cause our reading code to assume additional messages are present. **Note:** This restriction also holds for using `writev`, in which case each of the individual blocks must also be a multiple of 512 in size. **Note:** The work of aggregating smaller blocks into a single in-memory block prior to writing slows us down as much as not using `O_DIRECT` mode. **Note:** This was handled in the API by enabling `StreamPackets` to be written sequentially into the same large buffer; the plan is to write that buffer when it becomes full and copy the remaining unwritten bytes into a new buffer ready to be filled, then flush the (zero-padded) empty buffer at the end of writing.

TRIM: SSD drives must explicitly erase old blocks before re-use, which sometimes requires consolidation of information from partial blocks. If this is not done before the buffers are needed for new writes, writing performance can slow down. On Linux, the *fstrim* command can be used to manually clear out old buffers. Also, the *discard* option should not be added to the */etc/fstab* entry that mounts the drive, it causes blocks to be trimmed when a file is deleted. Disabling trimming except weekly can be done using the *sudo systemctl enable --now fstrim.timer* command.

Separate disks: Writing to four separate non-RAID disks is faster than writing to the hardware RAID array when we are not using O_DIRECT mode.

Disable journaling: It is possible to disable journaling on the file system used on SSD, which will lose data if the system loses power but may increase the write speed.

Prefetch: When reading data from disk that will be sent over the network at a metered rate, packets can be loaded from disk into memory buffers ahead of when they are needed to send. This hides the disk-read latency.

See the appendix for how to configure the Linux RAID drivers on a new kernel build.

Network tuning

Attached router: When using an XSight display that sends multicast packets to the Render Server, the receiving Ubuntu server must be attached to a router. That router does not need to be connected to anything else (or to the Internet), but having it provides multicast support that enables the host to then receive multicast packets on all its NICs. For some configurations, the NIC address 0.0.0.0 must be used rather than the address of the NIC receiving packets.

Cabling and reboot: Testing on 8/29/2024 found 60Hz 25-camera streaming working from the Storage host to the Render host, but missing packets when sending the other direction even for 18-21 cameras. Switching the ends of the fiber cable between hosts required rebooting to re-establish all configuration settings (the cable swap did seem to improve things, but not solve the problem). Once reboot was performed, data could stream both directions without loss.

Nvidia drivers: NVidia has optimized drivers for the Mellanox network cards that are used on the Storage Server. The drivers for Linux are available at the link in the footnote here¹ and for Windows here². They provide much higher performance than the generic drivers supplied with the operating system. They provide a **guide for performance tuning** here³. It recommends the following:

- Interrupt moderation: Reducing the number of interrupts to the CPU cores by waiting for multiple packets to arrive using counters and timers as described here⁴. This is dynamically tuned by default as described here⁵. (See the *interrupt coalescence*) section below for what was tried.
- (not done) Receive packet steering: Selects the CPU to perform protocol processing on top of the interrupt handler as described here⁶.
- (not done) Linux sysctl tuning: As described here⁷, these mostly refer to TCP-based tuning. This looks like an alternate interface to also control the OS buffer pool sizes as described below under *Global socket * size*.
- (not done): BIOS adjustment of the processor P and C states to disable sleeping and disabling hyperthreading and IO non-posted prefetching.

¹ https://network.nvidia.com/products/ethernet-drivers/linux/mlnx_en

² <https://network.nvidia.com/products/adaptersoftware/ethernet/windows/winof-2>

³ <https://enterprise-support.nvidia.com/s/article/performance-tuning-for-mellanox-adapters>

⁴ <https://enterprise-support.nvidia.com/s/article/understanding-interrupt-moderation>

⁵ <https://enterprise-support.nvidia.com/s/article/dynamically-tuned-interrupt-moderation--dim-x>

⁶ <https://enterprise-support.nvidia.com/s/article/receive-packet-steering>

⁷ <https://enterprise-support.nvidia.com/s/article/linux-sysctl-tuning>

- (not done) Use the CPU cores directly connected to the relevant PCIe bus where the Mellanox adapter is located.

Configuration under Windows 11 included the following:

- In *Device Manager* under *Network adapters*, pick the connected *Mellanox ConnectX-6 Dx Adapter*; right click and open *Properties*. Under *Advanced*, set *Jumbo Packet* to **9014**.
- In *Device Manager* under *Network adapters*, pick the connected *Mellanox ConnectX-6 Dx Adapter*; right click and open *Properties*. Under *Advanced*, set *Receive Buffers* to **4096** (for the ATTO Thunderlink, set this to 4000).
- (Fixed in the more recent adapter driver available in April 2025.) The installer for the Mellanox network card appears to be broken. One approach is to extract just the driver files from the EXE blob and then tell Windows to update the old driver using these extracted files.
- (Fixed in the more recent adapter driver available in April 2025.) The IPV4 address cannot be configured using the Windows 11 interface. The older approach works: Control Panel, Network, Change adapter settings (ends up in Network Connections), right-click Properties (on selected interface). To navigate through the confusing Windows 11 interface: Settings; Network & Internet; Advanced network settings; Select interface; More adapter options; Edit.
- For the **ATTO Thunderlink, (Windows)**
 - Ensure that the link speed is 50-100Gigabits; the default seems to set to 25. Do this in the Device Manager by changing the Advanced setting of *Speed & Duplex* from *Auto Negotiation* to *50 Gbps Full Duplex*.
 - Disable quality of service, also in Advanced.
 - Consider disabling interrupt moderation, also in Advanced.
 - ~~Also add a new registry key (DWORD):~~
~~HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Afd\Parameters~~
~~*—EnableDynamicBacklog=1~~
- Switch the network to Private so that the firewall can be turned off. If the GUI approach does not work, the following PowerShell method can be used:
 - Open PowerShell as administrator.
 - `get-netconnectionprofile` (lists all interfaces that are connected and active). For this situation, since the 100GigE interface cannot access the Internet, the name is "Unidentified network"
 - `set-netconnectionprofile -name "Unidentified network" -network private`
- There is documentation on tuning the cards at <https://docs.nvidia.com/networking/display/winofv55053000/general+performance+optimization+and+tuning> under "Performance Tuning and Counters".

Configuration under Linux for the ATTO Thunderlink included the following:

- Run `sudo apt install thunderbolt-tools` to get the Intel Thunderbolt tools. After the device is plugged in run `fwupdmgr get-devices` to detect the device; after that, the blue lights come on and running `boltctl list` will show the device.
- **The device must be plugged in when Linux boots for its network devices to be visible.**
- Run `sudo ./network_tuning_ATTO.sh` in the ASDP_Core_API repository to configure the cards for 50Gbits/second full duplex and set other required network settings.
- Be sure that the network mask on all systems is the same, it sets the broadcast address.

MTU size: The maximum transfer unit (MTU) size for Ethernet is 1500 bytes by default. Using "jumbo packets" with MTU size 9000 decreases the overhead for transmissions and enables higher throughput. To set this on device DEV, use **`sudo ifconfig DEV mtu 9000`** on the host where **DEV** is the name of the network device. The Core API library uses an MTU of 9000, so this must be configured on all network cards in the system.

Mellanox tuning: A set of tools for tuning Mellanox parameters is available here⁸. This Mellanox git repository includes a python subdirectory that has a *mlnx_tune* utility. When run as root, it configures the system for one of the specified profiles (use *sudo python3 mlnx_tune -h* to get a list of the profiles. Two likely choices are HIGH_THROUGHPUT and IP_FORWARDING_MULTI_STREAM_0_LOSS. We ran this command to configure HIGH_THROUGHPUT on all systems where we were able to get full-rate receiving to work reliably: *sudo python3 mlnx_tune -p HIGH_THROUGHPUT* so it may be required once on each system and then the system rebooted.

Global receive buffer size: There is a system-wide number of available buffers. Using *sudo sysctl -w net.core.rmem_max=26214400* (25MB) on the client enabled 25-camera (5 sending threads) operation at 125fps for 180 seconds without dropped packets.

Global UDP buffer pool size: There is also an operating-specified limit on the total number of pages available for UDP buffers. On Linux, the *net.ipv4.udp_mem* (readable via */proc/sys/net/ipv4/udp_mem*) sets the number of 4K blocks available system wide. This can be adjusted using *sysctl* and is set by default based on the total RAM size at boot time. When reaching 125fps without dropped packets, this was set on Linux using: *sudo sysctl -w net.ipv4.udp_mem=4000000*

Global max backlog: This was not needed to achieve full throughput in our systems, but it is recommended for high-throughput systems: *sysctl -w net.core.netdev_max_backlog=3000* or to make it permanent: *echo 'net.core.netdev_max_backlog = 3000' >> /etc/sysctl.conf* (one site recommends setting it to 30000 from its default value of 1000; other sites recommend setting it to 2000).

UDP socket read buffer size: There is a pre-specified buffer for incoming data on each UDP socket, which can cause overflow when the client does not read fast enough to pull every packet before the next one arrives. This buffer size can be set using the *SO_RECVBUF* socket option using *setsockopt()*. Setting it to be large enough to handle multiple incoming packets enables the system to be robust to longer periods between socket servicing.

UDP socket send buffer size: Controlled by *SO_SNDBUF* option in *setsockopt()*. This avoids packets being dropped before they are sent because the thread queues up a second one before the first has been transmitted.

Interrupt coalescence: The *ethtool* command can be used to adjust the maximum time to wait and the maximum number of buffers to fill before generating an interrupt. When tested across a range of rx-usecs (8, 32, 128, 256) and a range of rx-frames (16, 64, 128) there was not much difference in an application that was seeing a few lost frames on some cameras. The initial setting of 32, 64 was as good or better than the others in at least one of the two sweeps made of the parameters. It is likely that the Mellanox tuning set these parameters for high throughput when it was run. The *check_coalescing.sh* script in the *ASDP_Core_API* library can be used to re-test this if needed.

Open Fabric: NVidia has a set of “open fabric” drivers that can provide a unified address space among 100GigE-connected hosts and their GPUs that handles virtual memory among all devices. If needed, this could be used for transferring data from the Storage Server to the Render Server.⁹

network_tuning.sh: This script lives in the *ASDP_Core_API* directory. It can be copied into */usr/local/bin* and made executable and then add the following to the file */etc/systemd/system/network_tuning.service* and then run *sudo systemctl daemon-reload* and *sudo systemctl enable my-startup.service* :

```
[Unit]
Description=ASDP Network Tuning
Wants=network.target
After=network.target
```

⁸ <https://github.com/Mellanox/mlnx-tools>

⁹ <https://docs.nvidia.com/networking/display/connectx6dxen/linux+driver+installation>

```
[Service]
ExecStart=/usr/local/bin/network_tuning.sh
Type=oneshot
RemainAfterExit=yes
```

```
[Install]
WantedBy=multi-user.target
```

Buffer copy tuning

Pre-allocation: In situations where there is the need to pass large buffers from one thread to another (network read to disk write, for example), consider using a pool of available vectors that is pre-allocated before the program starts to be able to cover the expected load (additional ones can be allocated at run time if needed). The producer can then mark these are being used and the consumer can mark them as being available. This avoids the need to copy from a just-read buffer into one that will be passed to the consumer; the producer can write directly into an available buffer and pass it without copying to the consumer. This allocation and handling the free buffer recycling can be done in a buffer-allocator class that is used by both the producer and consumer.

Spin-free queues: When passing buffers from one thread to another to be sent to disk, network, or GPU it is helpful for neither the sender to have to spin waiting to queue (or waiting for the queue to drain to a specified level) nor the receiver needing to spin waiting for new elements in the queue. A helper class that uses condition variables to support waiting for queues to be empty enough or full enough without spin-waiting.

GPU sends: To reduce the overhead of sending data to the GPU, network data packets are accumulated in batches and then sent to the asynchronous copy and then kernel run. On Linux, a batch size of 16 (48 lines) is sufficient. Windows requires a batch size of greater than 64 for 21 cameras and more like 300 for 25 cameras. To minimize the size of the last-sent batch, the value of 110 packets (330 lines) will be used – this will send at 330, 660, and 990 and leave a small batch for the last one, reducing latency. For 25 cameras, 330 packets (990 lines) will be sent in the first and then the small remainder in the second.

GPU threads: A single thread sending all streams to the GPU was insufficient on both Windows and Linux. Linux requires 2 threads for 21 cameras and 3 for 25. Windows requires 2 threads for each (3 slows things down for 25 cameras).

Appendix A: Installing RAID drivers

When a new kernel is built under Linux (after ***sudo apt upgrade***, for example), the storage server does not properly install the RAID drivers required to access the four LLVM drives used to store data. It turns out that ***the script that installs the drivers fails when there is more than one kernel installed***. To avoid a failure to boot during this time, do the following:

- Comment out the `/mnt/storage` line in `/etc/fstab` before rebooting with multiple kernels.
- Reboot and verify that the new kernel works with all devices other than the RAID array.
- Remove the older kernel packages using ***sudo apt purge***, getting the headers, image, modules, and modules-extra packages.
- Uninstall the previous drivers using ***sudo /usr/sbin/hptuninhptnvme***
- Reboot the system using ***sudo reboot***
- Install the drivers from the executable in the jmott home directory:
 - o ***sudo /home/jmott/RAID_driver/hptnvme_g5_linux_src_v1.6.14.0_2023_12_26.bin***
- Reboot the system using ***sudo reboot***
- Verify that the device `/dev/hptblock0n0p` exists: ***ls /dev/hptblock0n0p***
- Uncomment the `/mnt/storage` line in `/etc/fstab`
- Test mounting the drive using ***sudo mount /mnt/storage***

Appendix B: Storage Server fails to boot

Sometimes after the storage server adds a new kernel, its boot cycle gets confused; **the default boot menu takes you back to another boot menu, and selecting in that one causes the kernel to fail to load.**

The solution is to select advanced Ubuntu boot options and pick the previous kernel. Once it has booted, use

- `sudo update-initramfs -u`
- `sudo reboot`

This should build all required kernels and let the normal boot sequence work with the latest kernel.